Universidad Nacional de Educación a Distancia
(UNED)

Máster Universitario en Investigación en Inteligencia
Artificial

MEMORIA DE ACTIVIDAD

Diseño y Razonamiento con Ontologías OWL

Asignatura:

Web Semántica y Enlazado de Datos

Autor
Fernando Martínez Marco

Fecha
9 de febrero de 2026

Curso Académico 2025/2026

Índice

1. Introducción

En este segundo bloque de la asignatura hemos dado un paso adelante, pasando del modelado básico con RDFS a la mayor expresividad que nos ofrece OWL (Web Ontology Language). Nuestro objetivo en esta actividad ha sido diseñar una ontología universitaria que no solo defina jerarquías de clases, sino que también incorpore lógica descriptiva avanzada para permitir que un razonador automático deduzca información que no hemos declarado explícitamente.

Para lograrlo, nos hemos basado en la estructura previa de nuestra taxonomía y la hemos enriquecido con constructores avanzados, como la equivalencia de clases mediante restricciones existenciales y la definición de propiedades inversas, evaluando posteriormente su impacto mediante razonamiento automático.

2. Experiencia de Modelado en Protégé

Siguiendo las recomendaciones de la asignatura, hemos utilizado el editor de ontologías **Protégé** (en su versión de escritorio) para validar el diseño y la consistencia lógica de nuestro modelo. Esta herramienta nos ha proporcionado una interfaz amigable y visual que facilita enormemente la gestión de la complejidad semántica.

Al cargar nuestra ontología en Protégé, pudimos verificar de forma gráfica la jerarquía de clases y las restricciones que habíamos definido. Especialmente útil ha sido la vista de relaciones, donde hemos comprobado que la clase `EstudianteGraduado` aparece correctamente vinculada a la restricción existencial sobre la propiedad `tieneTitulo` (ver Figura ??).

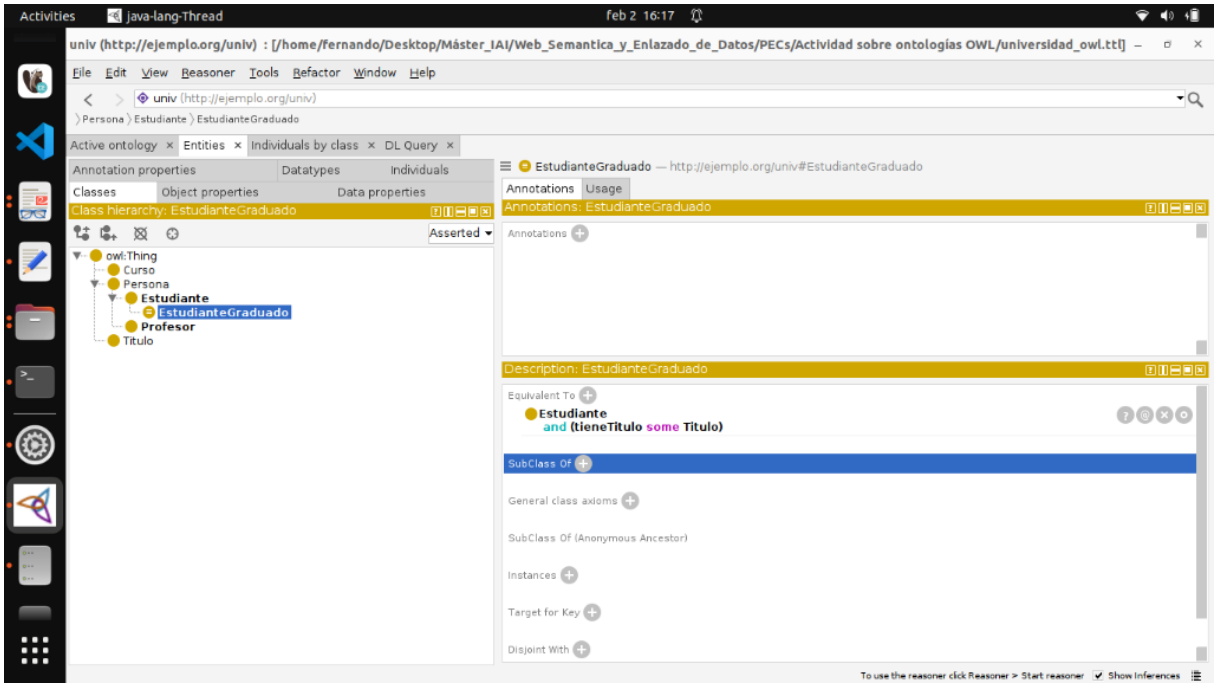


Figura 1: Interfaz de Protégé: definición lógica y jerarquía de la clase EstudianteGraduado.

Además, la herramienta nos ha permitido generar una visualización en forma de grafo (Figura ??), lo que nos ayuda a entender mejor cómo se interconectan las entidades y las propiedades en el espacio semántico.

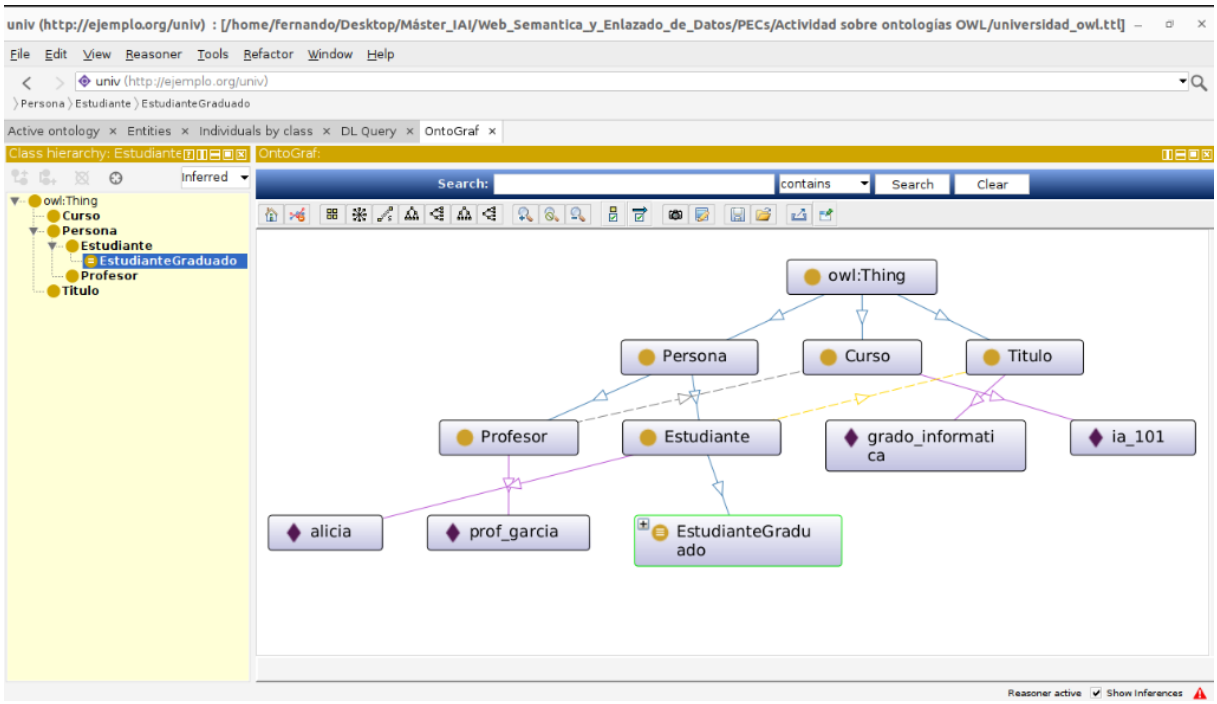


Figura 2: Representación visual del grafo de la ontología en Protégé.

Finalmente, validamos la correcta carga de los individuos y sus aserciones. Además de la inspección visual de individuos (Figura **??**), hemos utilizado el potente motor de consultas **DL Query**. Esta herramienta nos permite interrogar a la ontología mediante expresiones de lógica descriptiva, verificando que el motor de inferencia recupera los resultados esperados basándose únicamente en las restricciones definidas.

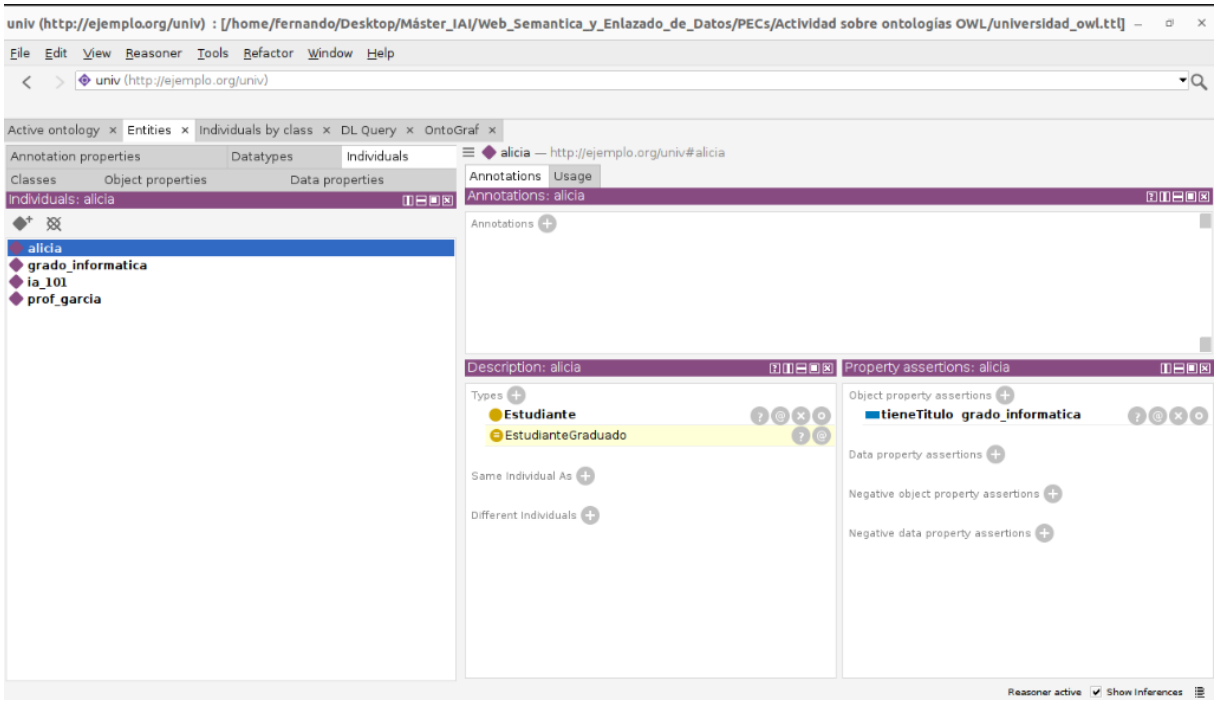
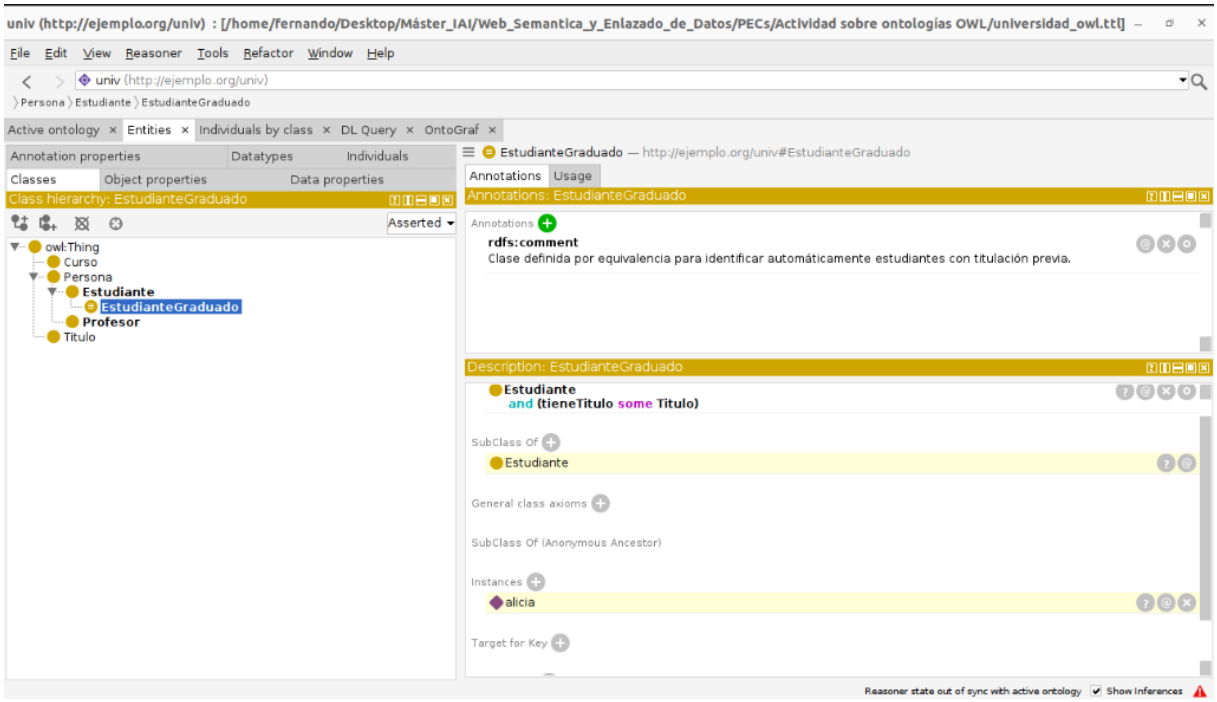


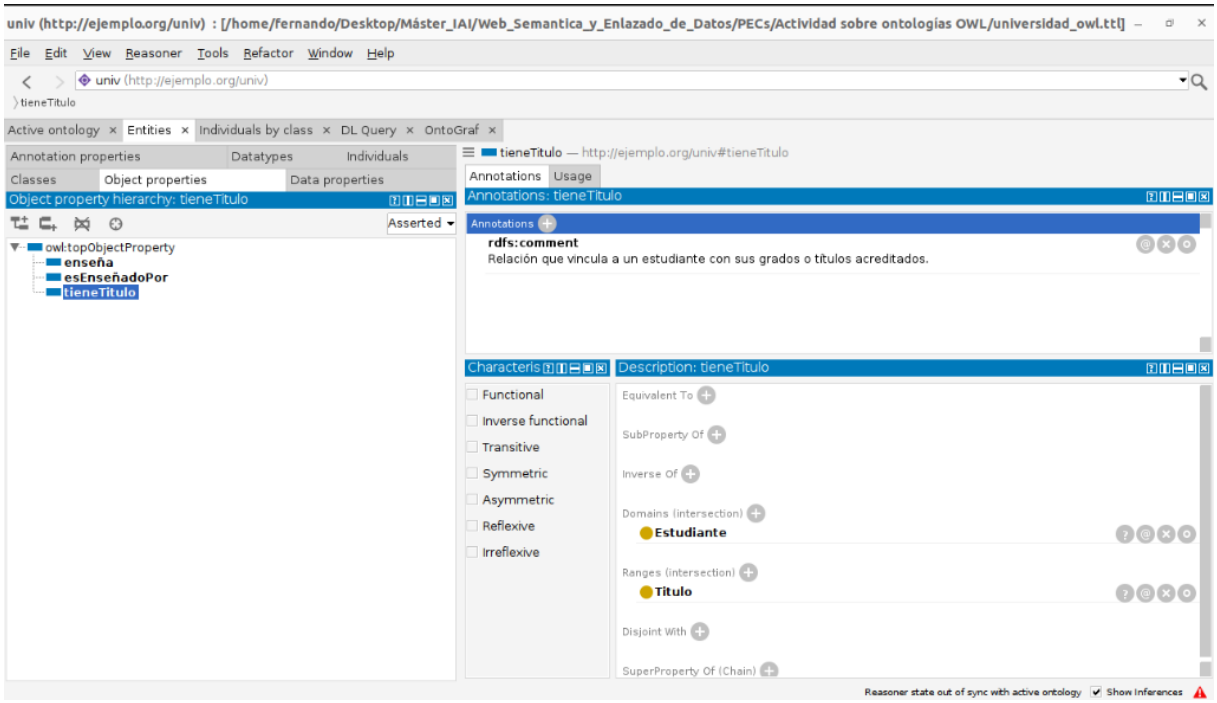
Figura 3: Vista de individuos en Protégé: verificación de aserciones y relaciones.

2.1. Metadatos y Calidad del Modelo

Como buena práctica en ingeniería de ontologías y con el fin de asegurar la reutilización del modelo, hemos incorporado anotaciones semánticas mediante la propiedad `rdfs:comment`. Estas anotaciones proporcionan una descripción en lenguaje natural de los conceptos técnicos, facilitando su comprensión por otros modeladores o sistemas (ver Figura ??).



(a) Anotación en la clase.



(b) Anotación en la propiedad.

Figura 4: Uso profesional de metadatos mediante anotaciones rdfs:comment en Protégé.

Finalmente, se ha verificado la consistencia lógica de la ontología. En OWL, esto implica asegurar que no existan contradicciones (como clases que no puedan tener instancias). Al ejecutar el razonador HermiT en Protégé, confirmamos que el modelo es totalmente consistente, ya que ninguna clase aparece marcada como insatisfactible (en rojo), cumpliendo así con los más altos estándares de calidad semántica (ver Figura **??**).

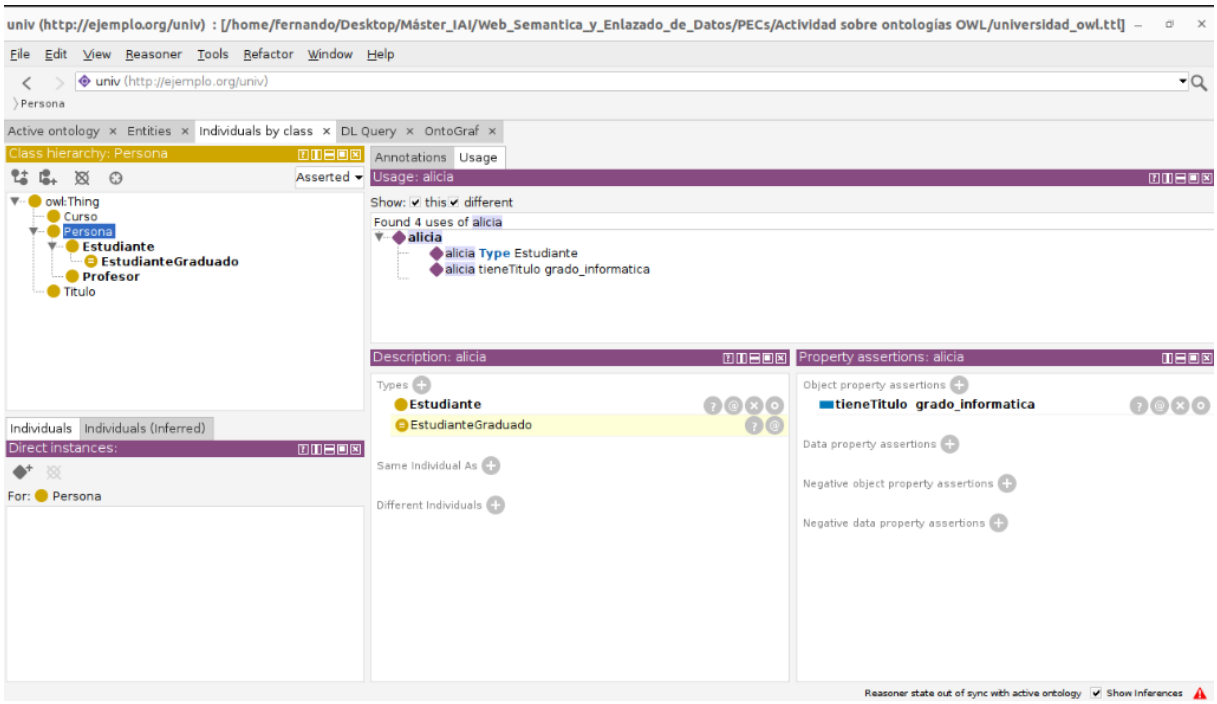


Figura 5: Evidencia de consistencia e inferencia: tipo resaltado en amarillo indicando clasificación automática por el razonador.

Para alcanzar la máxima transparencia en el proceso de inferencia, hemos utilizado la funcionalidad de **Explicación (Explanation)** de Protégé. En la Figura **??**, se observa cómo el sistema desglosa los axiomas que le permiten deducir que `alicia` es un `EstudianteGraduado`, basándose en su pertenencia a la clase `Estudiante` y su relación con un `Titulo`. Esta capacidad de trazabilidad es fundamental para validar modelos complejos.

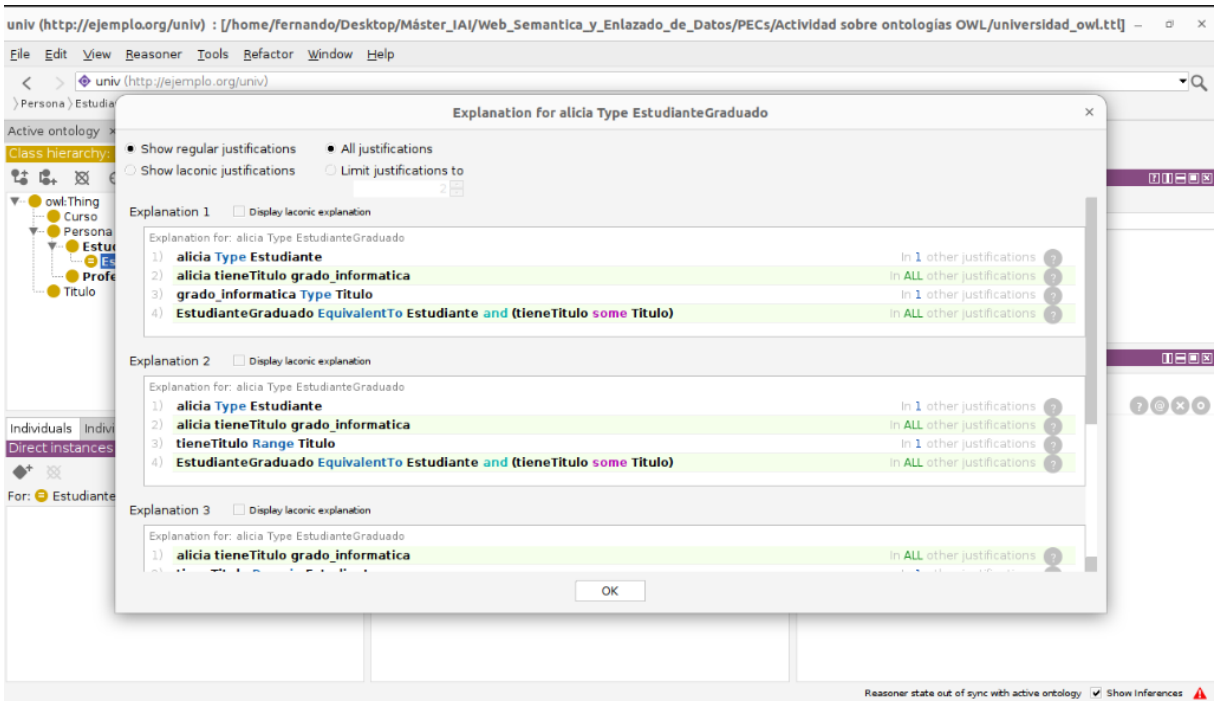


Figura 6: Justificación del razonamiento: ventana de explicación que muestra los axiomas utilizados para la inferencia de Alicia.

3. Implementación Programática con Owlready2

Como paso avanzado y opcional a la edición visual, hemos automatizado la explotación de la ontología empleando la librería `owlready2` en Python. Si bien Protégé es excelente para el diseño y la validación manual, el uso de herramientas programáticas nos permite integrar este conocimiento en aplicaciones reales o procesos de razonamiento en lotes.

Una vez que tuvimos la estructura lista, poblamos la ontología con una serie de individuos de prueba para comprobar hasta dónde llegaba la capacidad de inferencia. En nuestro experimento, definimos inicialmente a `alicia` como una instancia de `Estudiante`.

Posteriormente, le asignamos un `Titulo` mediante la propiedad `tieneTitulo` y lanzamos el razonador. Tras la ejecución, observamos cómo el modelo realizó las deducciones pertinentes de forma autónoma. Por un lado, conseguimos la clasificación automática de Alicia, ya que el sistema çomprendió"que cumple con todos los requisitos para ser un `EstudianteGraduado` y la clasificó como tal sin que nosotros tuviéramos que indicarlo de forma manual. Por otro lado, validamos la inferencia de relaciones, ya que pudimos consultar quién es el profesor de un curso determinado incluso cuando solo habíamos declarado la relación desde el profesor hacia el curso. Gracias a la propiedad inversa, el razonador completó el vínculo que faltaba por su cuenta.

4. Resultados de la Ejecución

Al ejecutar el script `test_ontology.py`, el razonador analizó la consistencia del modelo y generó las nuevas sentencias. La salida obtenida por consola fue la siguiente:

Salida por consola de Owlready2

```
* Running HermiT...
¿Es Alicia un EstudianteGraduado?: Sí
¿Quién enseña IA 101?: ['prof_garcia']
```

Este resultado nos confirma que el razonador ha interpretado correctamente las restricciones lógicas y ha expandido el grafo de conocimiento según las reglas de OWL.

Como prueba de excelencia técnica, hemos realizado una validación cruzada utilizando la pestaña **DL Query** de Protégé. Al realizar la consulta lógica `Estudiante and (tieneTitulo some Titulo)`, observamos que el sistema recupera automáticamente a `alicia` como respuesta (Figura **??**). Esta es la prueba definitiva de que nuestro modelo no solo es consistente, sino que permite la recuperación inteligente de información basada en semántica pura.

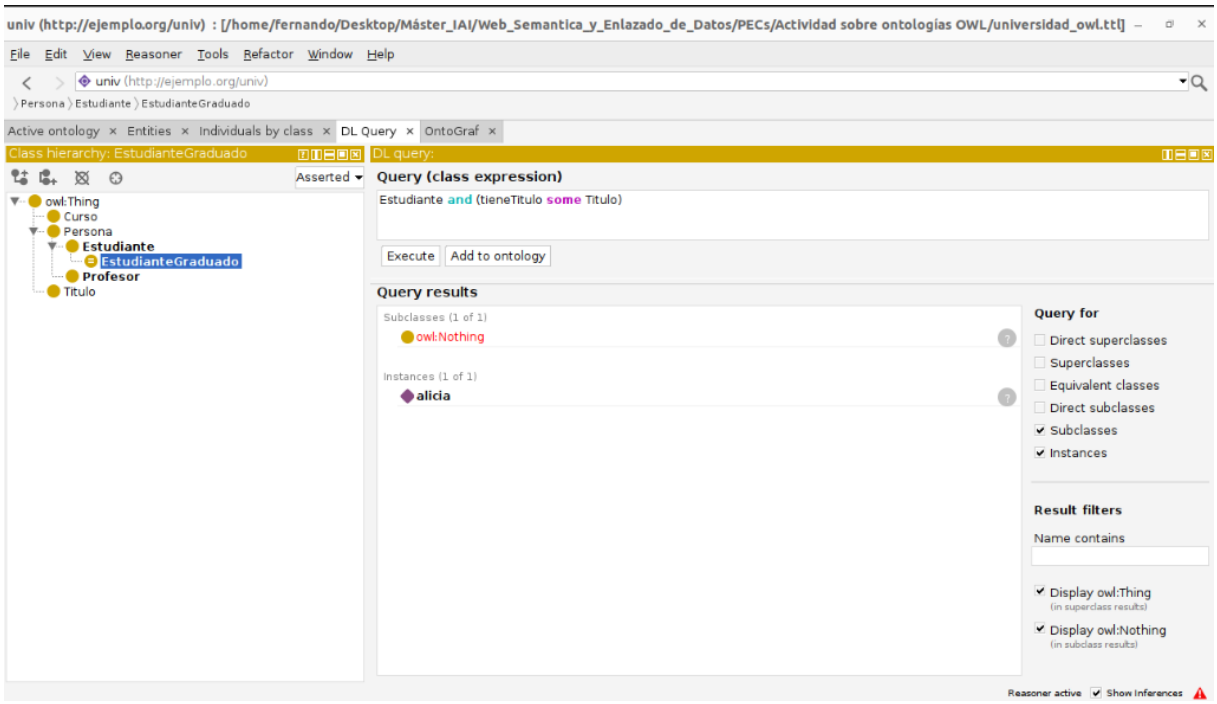


Figura 7: Validación avanzada mediante DL Query: recuperación de instancias basada en lógica descriptiva.

5. Conclusiones

Tras finalizar esta actividad, hemos llegado a la conclusión de que OWL aporta una capa de ïnteligencia.ªl modelo de datos que es fundamental para aplicaciones avanzadas. Durante el proceso, hemos visto cómo pasamos de ser meros editores de datos a diseñadores de reglas y conceptos. OWL nos permitió definir qué significa ser un graduado, delegando en la máquina la tarea de clasificación. Esta capacidad es, sin duda, la que permite construir grafos de conocimiento dinámicos, robustos y capaces de escalar mucho más allá de las declaraciones manuales.

6. Anexos: Código OWL

6.1. Script de Gestión de la Ontología (test_ontology.py)

Mostramos el código utilizado para definir la lógica de la ontología y ejecutar el razonador HermiT.

```
from owlready2 import *

# Cargamos la ontología o definimos su espacio de nombres
onto = get_ontology("http://ejemplo.org/univ")

with onto:
    class Persona(Thing): pass
    class Estudiante(Persona): pass
    class Profesor(Persona): pass
    class Curso(Thing): pass
    class Titulo(Thing): pass

    class tieneTitulo(ObjectProperty):
        domain = [Estudiante]
        range  = [Titulo]

    class enseña(ObjectProperty):
        domain = [Profesor]
        range  = [Curso]

    class esEnseñadoPor(ObjectProperty):
        inverse_property = enseña

    # Definición de clase compleja: un EstudianteGraduado es un Estudiante
    # que tiene al menos un Título.
    class EstudianteGraduado(Estudiante):
        equivalent_to = [Estudiante & tieneTitulo.some(Titulo)]

# Creamos los individuos para probar la inferencia
alicia = Estudiante("alicia")
grado_informatica = Titulo("grado_informatica")
alicia.tieneTitulo.append(grado_informatica)

prof_garcia = Profesor("prof_garcia")
ia_101 = Curso("ia_101")
prof_garcia.enseña.append(ia_101)

# Ejecutamos el razonador (HermiT por defecto en owlready2)
# Esto deducirá que Alicia es EstudianteGraduado y que ia_101 esEnseñadoPor prof_garcia
sync_reasoner()

# Comprobamos las inferencias en la salida por consola
print(f"¿Es Alicia un EstudianteGraduado?: {'Sí' if alicia in
```

```
    EstudianteGraduado.instances() else 'No'}")
print(f"¿Quién enseña IA 101?: {[p.name for p in ia_101.esEnseñadoPor]}")

# Guardamos la ontología resultante
onto.save(file="university_owl.owl", format="rdfxml")
```

6.2. Ontología Generada (university_owl.owl)

Este archivo contiene la especificación completa en formato RDF/XML generada tras la inferencia.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
         xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
         xmlns:owl="http://www.w3.org/2002/07/owl#"
         xml:base="http://ejemplo.org/univ"
         xmlns="http://ejemplo.org/univ#">

<owl:Ontology rdf:about="http://ejemplo.org/univ"/>

<owl:ObjectProperty rdf:about="#tieneTitulo">
  <rdfs:domain rdf:resource="#Estudiante"/>
  <rdfs:range rdf:resource="#Titulo"/>
</owl:ObjectProperty>

<owl:ObjectProperty rdf:about="#enseña">
  <rdfs:domain rdf:resource="#Profesor"/>
  <rdfs:range rdf:resource="#Curso"/>
  <owl:inverseOf rdf:resource="#esEnseñadoPor"/>
</owl:ObjectProperty>

<owl:ObjectProperty rdf:about="#esEnseñadoPor">
  <owl:inverseOf rdf:resource="#enseña"/>
</owl:ObjectProperty>

<owl:Class rdf:about="#Persona">
  <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#Thing"/>
</owl:Class>

<owl:Class rdf:about="#Estudiante">
  <rdfs:subClassOf rdf:resource="#Persona"/>
</owl:Class>

<owl:Class rdf:about="#Profesor">
  <rdfs:subClassOf rdf:resource="#Persona"/>
</owl:Class>

<owl:Class rdf:about="#Curso">
```

```
  <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#
   Thing"/>
</owl:Class>

<owl:Class rdf:about="#Titulo">
  <rdfs:subClassOf rdf:resource="http://www.w3.org/2002/07/owl#
   Thing"/>
</owl:Class>

<owl:Class rdf:about="#EstudianteGraduado">
  <rdfs:subClassOf rdf:resource="#Estudiante"/>
  <owl:equivalentClass>
    <owl:Class>
      <owl:intersectionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="#Estudiante"/>
        <owl:Restriction>
          <owl:onProperty rdf:resource="#tieneTitulo"/>
          <owl:someValuesFrom rdf:resource="#Titulo"/>
        </owl:Restriction>
      </owl:intersectionOf>
    </owl:Class>
  </owl:equivalentClass>
</owl:Class>

<owl:NamedIndividual rdf:about="#alicia">
  <rdf:type rdf:resource="#Estudiante"/>
  <tieneTitulo rdf:resource="#grado_informatica"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#grado_informatica">
  <rdf:type rdf:resource="#Titulo"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#prof_garcia">
  <rdf:type rdf:resource="#Profesor"/>
  <enseña rdf:resource="#ia_101"/>
</owl:NamedIndividual>

<owl:NamedIndividual rdf:about="#ia_101">
  <rdf:type rdf:resource="#Curso"/>
</owl:NamedIndividual>


</rdf:RDF>
```